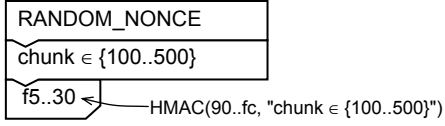


Above is an example of the simplest possible macaroon credential. If it grants bearers access to a key-value storage service for chunks, it can be attenuated by adding a caveat whose predicate restricts accessible chunks, as below:



To check the macaroons' integrity, the storage service can compute and verify the terminal, derived HMAC value, even when the resulting macaroon is further attenuated with a caveat whose predicate permits only reading, as below:

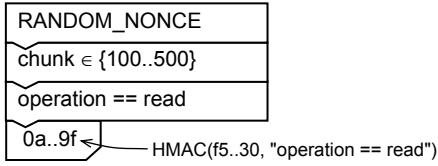


Fig. 3. Three macaroons, respectively giving full access to all chunks at a key-value storage service, permitting all requests to chunks in the range 100 to 500, or allowing only reading from that range. Starting at its key K_{TS} , the service can verify the integrity of each macaroon by deriving its chain of nested HMAC values. Macaroons are presented with requests, and the service permits only requests that satisfy the macaroons' caveats.

private images at TS . From this URL, the owner can create a new, derived macaroon URL for read-only access to an image, embed within it a third-party caveat that requires proof of membership in a group G at service A , and share it via email. Any recipient of this derived URL would have to get a fresh discharge for the third-party caveat—which may require the user to log into A , via a redirection—and pass it along to TS when using the URL to view the image. All of the above could be accomplished using software, such that both parties remained unaware of the use of macaroons.

III. CONCRETE MECHANISMS AND EXAMPLES

Concretely, a macaroon granting authority to a Cloud server is constructed from a sequence of messages and a chain of keyed cryptographic digests derived from those messages. This chain of digest values is computed using HMAC functions with appropriate truncation (e.g., to 128 bits) [43]. Each macaroon contains, however, only the terminal, final HMAC value in the chain, with this value serving as the macaroon's *signature*: a generalized message authentication code that (i) allows target services to verify the integrity of macaroons, and (ii) also allows shared keys to be established with the bearers of any derived macaroons.

Fundamental to macaroons is how this chain of keyed HMAC values is constructed, in a nested fashion, along the sequence of messages in a macaroon—of which, all but the first are caveats. The first message in a macaroon is required to be a public, opaque *key identifier* that maps to a secret *root key* known only to the target service. (Such key identifiers can be implemented, e.g., using random nonces, indices into a database at the target service, keyed HMACs, or using public-key or secret-key encryption; see [37, Sections 4.2 and 4.3]

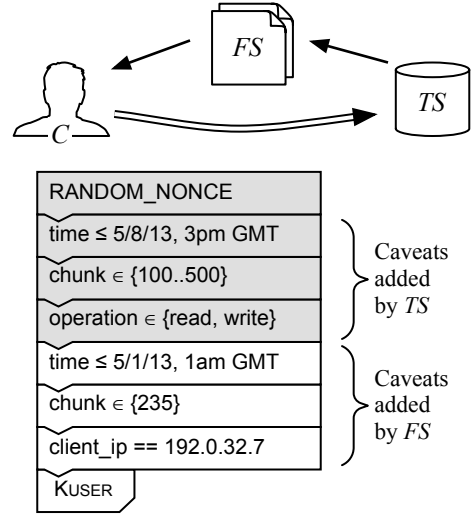


Fig. 4. A macaroon held by the user at client C , authorizing limited-time access to chunk 235 at a key-value storage service TS from one network IP address. This macaroon is derived from another (shown shaded in gray) that a social-networking “forum service” FS holds for a range of chunks at TS —delegating and attenuating the authority of that macaroon.

for a discussion.) Starting with this root key, a macaroon's chain of HMAC values is derived by computing, for each message, a keyed HMAC value, nested so that each such value itself is used as the HMAC key for the next message. Thus, a macaroon with two messages $MsgId$ and $MsgCav$ and root key K_R could have the signature $HMAC(K_{tmp}, MsgCav)$, where K_{tmp} is $HMAC(K_R, MsgId)$.

An example of this concrete macaroon construction is shown in Figure 3. (This example, like others in this paper, draws from the distributed systems literature where related chained-message-digest constructions have long been used for authorizing access to networked storage [3, 25, 51].)

In Figure 3, starting from a macaroon with a random nonce key identifier, and a secret key K_{TS} held at the target service, two macaroons are successively derived by adding caveat predicates and chaining the HMAC signatures. Each macaroon authorizes requests to a target service that offers key-value storage for *chunks*, with caveats in the derived macaroons restricting the authorized chunks and operations. To verify the integrity of these macaroons, the target service need only check their signature against the HMAC values 90..fc, f5..30, or 0a..9f, derived using its key K_{TS} .

This construction makes it particularly simple to delegate and attenuate authorization credentials, since the chain of HMAC signatures establishes shared keys between the target service and the bearers of successive, derived macaroons.

For example, Figure 4 shows a macaroon authorizing a user at client C to access chunk 235 at a key-value storage service TS . This macaroon has been derived as a strict attenuation of another, more permissive macaroon—held by the social-networking “forum service” FS —by both eliding that macaroon's signature (called, say K_{FS}), and by adding caveats whose predicates require a specific chunk and client IP address to be present in the request context. Even so, this macaroon's signature, K_{USER} , both allows the target storage service to verify its integrity, and also allows its bearer to